

See the Assessment Guide for information on how to interpret this report.

ASSESSMENT SUMMARY

Compilation: PASSED
API: PASSED

SpotBugs: PASSED
PMD: PASSED
Checkstyle: FAILED (0 errors, 10 warnings)

Correctness: 36/40 tests passed
Memory: No tests available for autograding.
Timing: 61/116 tests passed

Aggregate score: 83.52%
[Compilation: 5%, API: 5%, Style: 0%, Correctness: 70%, Memory: 20%]

ASSESSMENT DETAILS

The following files were submitted:

1.6K Nov 1 18:00 Inversions.java
1.7K Nov 1 18:00 MaximumSquareSubmatrix.java
1.1K Nov 1 18:00 Ramanujan.java

* COMPILING

% javac Inversions.java
*-----

% javac Ramanujan.java
*-----

% javac MaximumSquareSubmatrix.java
*-----

=====

Checking the APIs of your programs.
*-----
Inversions:

Ramanujan:

MaximumSquareSubmatrix:

=====

* CHECKING STYLE AND COMMON BUG PATTERNS

% spotbugs *.class
*-----

=====

% pmd .
*-----

=====

% checkstyle *.java
*-----
[WARN] Inversions.java:22:32: '++' is followed by whitespace. [NoWhitespaceAfter]
[WARN] Inversions.java:45:11: '//' or '/' is not followed by whitespace. [WhitespaceAfter]
[WARN] Ramanujan.java:15:15: '//' or '/' is not followed by whitespace. [WhitespaceAfter]
Checkstyle ends with 0 errors and 3 warnings.

% custom checkstyle checks for Inversions.java
*-----
[WARN] Inversions.java:7: The number (0) of calls to 'Long.parseLong()' must equal the number (1) of long integer command-line arguments. [CommandLineArgument]
[WARN] Inversions.java:7: The number (2) of calls to 'Integer.parseInt()' must equal the number (1) of integer command-line arguments. [CommandLineArgument]
Checkstyle ends with 0 errors and 2 warnings.

% custom checkstyle checks for Ramanujan.java
*-----
[WARN] Ramanujan.java:11:30: You should not need to call 'Math.pow()' in this program. Instead, you may wish to use 'Math.cbrt()'. [Design]
[WARN] Ramanujan.java:11:40: '3' looks like an unnecessary constant. [MagicNumber]
[WARN] Ramanujan.java:12:22: '1e-6' looks like an unnecessary constant. [MagicNumber]
[WARN] Ramanujan.java:14:29: You should not need to call 'Math.pow()' in this program. Instead, you may wish to use 'Math.cbrt()'. [Design]
[WARN] Ramanujan.java:14:47: '3' looks like an unnecessary constant. [MagicNumber]
Checkstyle ends with 0 errors and 5 warnings.

% custom checkstyle checks for MaximumSquareSubmatrix.java
*-----

=====

* TESTING CORRECTNESS

Testing correctness of Inversions
*-----
Running 16 total tests.

Test 1: check output format of main() for inputs from assignment specification
% java-introcs Inversions 10 0
0 1 2 3 4 5 6 7 8 9

% java-introcs Inversions 10 1
0 1 2 3 4 5 6 7 9 8

% java-introcs Inversions 10 45
9 8 7 6 5 4 3 2 1 0

% java-introcs Inversions 10 20
2 3 4 5 7 8 9 6 1 0

==> passed

Test 2: check correctness of main() for inputs from assignment specification
% java-introcs Inversions 10 0
% java-introcs Inversions 10 1
% java-introcs Inversions 10 45
% java-introcs Inversions 10 20
==> passed

```
Test 3: check generate() for fixed values of n and k
* n = 10, k = 0
* n = 10, k = 1
* n = 10, k = 45
* n = 10, k = 20
==> passed

Test 4: check generate() for k = 0
* n = 4, k = 0
* n = 5, k = 0
* n = 6, k = 0
* n = 8, k = 0
* n = 15, k = 0
* n = 20, k = 0
* n = 50, k = 0
* n = 64, k = 0
==> passed

Test 5: check generate() for k = n * (n-1) / 2
* n = 4, k = 6
* n = 5, k = 10
* n = 6, k = 15
* n = 8, k = 28
* n = 15, k = 105
* n = 20, k = 190
* n = 50, k = 1225
* n = 64, k = 2016
==> passed

Test 6: check count() for permutations of length 0, 1, 2, and 3
* a = { }
* a = { 0 }
* a = { 0, 1 }
* a = { 1, 0 }
* a = { 0, 1, 2 }
* a = { 0, 2, 1 }
* a = { 1, 0, 2 }
* a = { 1, 2, 0 }
* a = { 2, 0, 1 }
* a = { 2, 1, 0 }
==> passed

Test 7: check generate() for fixed n and random k
* 1000 random permutations of length 5, 0 <= k <= 10
* 1000 random permutations of length 6, 0 <= k <= 15
* 1000 random permutations of length 7, 0 <= k <= 21
* 1000 random permutations of length 10, 0 <= k <= 45
* 1000 random permutations of length 15, 0 <= k <= 105
* 1000 random permutations of length 20, 0 <= k <= 190
==> passed

Test 8: check count() for fixed permutations
* a = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
* a = { 1, 0, 2, 3, 4, 5, 6, 7, 8, 9 }
* a = { 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 }
* a = { 9, 8, 0, 1, 2, 3, 7, 4, 5, 6 }
==> passed

Test 9: check count() for sorted permutations of length n
* sorted permutation of length 4
* sorted permutation of length 5
* sorted permutation of length 6
* sorted permutation of length 8
* sorted permutation of length 15
* sorted permutation of length 20
* sorted permutation of length 50
* sorted permutation of length 64
==> passed

Test 10: check count() for reverse-sorted permutations of length n
* reverse-sorted permutation of length 4
* reverse-sorted permutation of length 5
* reverse-sorted permutation of length 6
* reverse-sorted permutation of length 8
* reverse-sorted permutation of length 15
* reverse-sorted permutation of length 20
* reverse-sorted permutation of length 50
* reverse-sorted permutation of length 64
==> passed

Test 11: check count() for permutations of length 0, 1, and 2
* a = { }
* a = { 0 }
* a = { 0, 1 }
* a = { 1, 0 }
* a = { 0, 1, 2 }
* a = { 1, 0, 2 }
* a = { 0, 2, 1 }
* a = { 2, 1, 0 }
==> passed

Test 12: check count() for all permutations of length n
* all permutations of length 4
* all permutations of length 5
* all permutations of length 6
* all permutations of length 7
* all permutations of length 8
==> passed

Test 13: check count() for random permutations of length n
* 1000 random permutations of length 9
* 1000 random permutations of length 10
* 1000 random permutations of length 12
* 1000 random permutations of length 15
* 1000 random permutations of length 20
* 1000 random permutations of length 25
==> passed

Test 14: check count() for random permutations of length n (larger n)
* 1000 random permutations of length 50
* 1000 random permutations of length 100
* 1000 random permutations of length 200
* 1000 random permutations of length 400
* 1000 random permutations of length 1000
==> passed

Test 15: check count() for permutations with huge number of inversions
* reverse-sorted permutation of length 65536
* reverse-sorted permutation of length 65537
  - student count() return wrong value
  - student count() = -2147450880
  - reference count() = 2147516416
  - a = { 65536, 65535, 65534, 65533, 65532, 65531, 65530, 65529, ...

* reverse-sorted permutation of length 131072
  - student count() return wrong value
  - student count() = -65536
  - reference count() = 8589869056
  - a = { 131071, 131070, 131069, 131068, 131067, 131066, 131065, ...

==> FAILED

Test 16: check that count() does not mutate argument array
* 10 random permutations of length 3
* 10 random permutations of length 5
* 10 random permutations of length 7
* 10 random permutations of length 10
==> passed

Inversions Total: 15/16 tests passed!
```

```
Testing correctness of Ramanujan
*-----
Running 15 total tests.

Test 1: check output format of main() with inputs from assignment specification
% java Ramanujan 1729
true

% java Ramanujan 3458
false

% java Ramanujan 4104
true

% java Ramanujan 216125
true

==> passed

Test 2: check correctness of main() with inputs from assignment specification
% java-introcs Ramanujan 1729
% java-introcs Ramanujan 3458
% java-introcs Ramanujan 4104
% java-introcs Ramanujan 216125
==> passed

Test 3: check isRamanujan() with arguments from assignment specification
* isRamanujan(1729)
* isRamanujan(3458)
* isRamanujan(4104)
* isRamanujan(216125)
==> passed

Test 4: check isRamanujan() with Ramanujan numbers
* isRamanujan(13832)
* isRamanujan(20683)
* isRamanujan(32832)
* isRamanujan(39312)
* isRamanujan(40033)
* isRamanujan(46683)
* isRamanujan(64232)
* isRamanujan(65728)
* isRamanujan(110656)
* isRamanujan(110808)
* isRamanujan(134379)
* isRamanujan(149389)
* isRamanujan(165464)
* isRamanujan(171288)
* isRamanujan(195841)
* isRamanujan(216027)
==> passed

Test 5: check isRamanujan() with non-Ramanujan numbers
* isRamanujan(1)
* isRamanujan(3)
* isRamanujan(4)
* isRamanujan(5)
* isRamanujan(93)
* isRamanujan(346)
* isRamanujan(51214)
==> passed

Test 6: check isRamanujan() with non-Ramanujan numbers
      (sum of two cubes in only 1 way)
* isRamanujan(2)
* isRamanujan(35)
  - reference isRamanujan(35) = false
  - student   isRamanujan(35) = true

* isRamanujan(65)
  - reference isRamanujan(65) = false
  - student   isRamanujan(65) = true

* isRamanujan(91)
  - reference isRamanujan(91) = false
  - student   isRamanujan(91) = true

* isRamanujan(128)
* isRamanujan(189)
  - reference isRamanujan(189) = false
  - student   isRamanujan(189) = true

* isRamanujan(341)
  - reference isRamanujan(341) = false
  - student   isRamanujan(341) = true

==> FAILED

Test 7: check isRamanujan() with non-positive integers
* isRamanujan(0)
* isRamanujan(-1)
* isRamanujan(-2)
* isRamanujan(-91)
* isRamanujan(-189)
* isRamanujan(-1729)
* isRamanujan(-123456789)
==> passed

Test 8: check isRamanujan() with Ramanujan triples
      (sum of two cubes in 3 different ways)
* isRamanujan(87539319)
* isRamanujan(119824488)
* isRamanujan(143604279)
* isRamanujan(175959000)
* isRamanujan(327763000)
* isRamanujan(700314552)
* isRamanujan(804360375)
* isRamanujan(958595904)
==> passed

Test 9: check isRamanujan() with Ramanujan quadruples
      (sum of two cubes in 4 different ways)
* isRamanujan(6963472309248)
* isRamanujan(12625136269928)
* isRamanujan(21131226514944)
* isRamanujan(26059452841000)
==> passed

Test 10: check isRamanujan() with random Ramanujan numbers
* 100 random Ramanujan numbers between 1 and 10000
* 10000 random Ramanujan numbers between 1 and 1000000
* 10000 random Ramanujan numbers between 1 and 10000000
* 10000 random Ramanujan numbers between 1 and 100000000
* 10000 random Ramanujan numbers between 1 and 1000000000
==> passed

Test 11: check isRamanujan() with random non-Ramanujan numbers
* 100 random non-Ramanujan numbers between 1 and 1000
  - reference isRamanujan(280) = false
  - student   isRamanujan(280) = true
  - x = 280
  - false positive on trial 19 of 100

* 10000 random non-Ramanujan numbers between 1 and 1000000
  - reference isRamanujan(713601) = false
  - student   isRamanujan(713601) = true
  - x = 713601
  - false positive on trial 610 of 10000

* 10000 random non-Ramanujan numbers between 1 and 10000000
  - reference isRamanujan(9577919) = false
  - student   isRamanujan(9577919) = true
  - x = 9577919
  - false positive on trial 225 of 10000
```



```
* 10000 random non-Ramanujan numbers between 1 and 100000000
- reference isRamanujan(69870347) = false
- student isRamanujan(69870347) = true
- x = 69870347
- false positive on trial 231 of 10000

* 10000 random non-Ramanujan numbers between 1 and 1000000000
- reference isRamanujan(975354626) = false
- student isRamanujan(975354626) = true
- x = 975354626
- false positive on trial 1113 of 10000

==> FAILED

Test 12: check isRamanujan() with random Ramanujan numbers (large integers)
* 100 random Ramanujan numbers between 1 and 10000000000
* 100 random Ramanujan numbers between 1 and 100000000000
* 100 random Ramanujan numbers between 1 and 1000000000000
* 100 random Ramanujan numbers between 1 and 10000000000000
* 100 random Ramanujan numbers between 1 and 100000000000000
* 100 random Ramanujan numbers between 1 and 1000000000000000
* 100 random Ramanujan numbers between 1 and 10000000000000000
==> passed

Test 13: check isRamanujan() with random non-Ramanujan numbers (large integers)
* 100 random non-Ramanujan numbers between 1 and 10000000000
* 100 random non-Ramanujan numbers between 1 and 100000000000
* 100 random non-Ramanujan numbers between 1 and 1000000000000
- reference isRamanujan(545571121295) = false
- student isRamanujan(545571121295) = true
- x = 545571121295
- false positive on trial 95 of 100

* 100 random non-Ramanujan numbers between 1 and 10000000000000
- reference isRamanujan(6190788789664) = false
- student isRamanujan(6190788789664) = true
- x = 6190788789664
- false positive on trial 21 of 100

* 100 random non-Ramanujan numbers between 1 and 100000000000000
- reference isRamanujan(19191933246374) = false
- student isRamanujan(19191933246374) = true
- x = 19191933246374
- false positive on trial 6 of 100

* 100 random non-Ramanujan numbers between 1 and 1000000000000000
- reference isRamanujan(674543817191593) = false
- student isRamanujan(674543817191593) = true
- x = 674543817191593
- false positive on trial 12 of 100

==> FAILED

Test 14: check isRamanujan() with Ramanujan numbers near 2^63 - 1
* isRamanujan(9223261564912520489)
* isRamanujan(9223278330318728221)
==> passed

Test 15: check that main() is consistent with isRamanujan()
* 10000 random Ramanujan numbers between 1 and 1000000
* 10000 random Ramanujan numbers between 1 and 10000000
* 10000 random Ramanujan numbers between 1 and 100000000
* 10000 random Ramanujan numbers between 1 and 1000000000
==> passed

Ramanujan Total: 12/15 tests passed!

=====
Testing correctness of MaximumSquareSubmatrix
*-----
Running 9 total tests.

Test 1: check output format for inputs from assignment specification
% java-introcs MaximumSquareSubmatrix < square5.txt
2

% java-introcs MaximumSquareSubmatrix < square6.txt
3

% java-introcs MaximumSquareSubmatrix < square7.txt
4

% java-introcs MaximumSquareSubmatrix < square10.txt
2

==> passed

Test 2: check that main() reads all data from standard input
% java-introcs MaximumSquareSubmatrix < square5.txt
% java-introcs MaximumSquareSubmatrix < square6.txt
% java-introcs MaximumSquareSubmatrix < square7.txt
% java-introcs MaximumSquareSubmatrix < square10.txt
==> passed

Test 3: check main() with input files from assignment specification
% java-introcs MaximumSquareSubmatrix < square5.txt
% java-introcs MaximumSquareSubmatrix < square6.txt
% java-introcs MaximumSquareSubmatrix < square7.txt
% java-introcs MaximumSquareSubmatrix < square10.txt
==> passed

Test 4: check size() on input files
* filename = square5.txt
* filename = square6.txt
* filename = square7.txt
* filename = square10.txt
==> passed

Test 5: check size() on random n-by-n matrices
* 1000 random 5-by-5 matrices, with each entry 1 with probability 0.5
* 1000 random 6-by-6 matrices, with each entry 1 with probability 0.1
* 1000 random 7-by-7 matrices, with each entry 1 with probability 0.9
* 1000 random 8-by-8 matrices, with each entry 1 with probability 0.99
* 1000 random 10-by-10 matrices, with each entry 1 with probability 0.5
* 1000 random 20-by-20 matrices, with each entry 1 with probability 0.1
* 1000 random 25-by-25 matrices, with each entry 1 with probability 0.9
* 1000 random 30-by-30 matrices, with each entry 1 with probability 0.99
==> passed

Test 6: check size() on n-by-n matrix of all 0s
* 1-by-1 matrix of all 0s
* 3-by-3 matrix of all 0s
* 5-by-5 matrix of all 0s
* 10-by-10 matrix of all 0s
* 15-by-15 matrix of all 0s
* 20-by-20 matrix of all 0s
==> passed

Test 7: check size() on n-by-n matrix of all 1s
* 1-by-1 matrix of all 1s
* 3-by-3 matrix of all 1s
* 5-by-5 matrix of all 1s
* 10-by-10 matrix of all 1s
* 15-by-15 matrix of all 1s
* 20-by-20 matrix of all 1s
==> passed

Test 8: check that size(a) does not mutate a[][] for random n-by-n matrices
* 10 random 3-by-3 matrices
* 10 random 5-by-5 matrices
* 10 random 7-by-7 matrices
* 10 random 10-by-10 matrices
```

==> passed

Test 9: check that main() is consistent with size()
* 1000 random 4-by-4 matrices
* 1000 random 5-by-5 matrices
* 1000 random 8-by-8 matrices
* 1000 random 10-by-10 matrices
==> passed

MaximumSquareSubmatrix Total: 9/9 tests passed!

=====

* TIMING

Timing Ramanujan
*-----

Running 44 total tests.

Tests 1-15: time isRamanujan(), for random non-Ramanujan numbers between 1 and 10^k,
for k = 4 to 18

The approximate order-of-growth is $n^{(\log \text{ratio})}$

	k	seconds	log ratio

=> passed	4	0.000	
=> passed	5	0.000	
=> passed	6	0.000	
=> passed	7	0.000	
=> passed	8	0.000	
=> passed	9	0.001	
=> passed	10	0.001	
=> passed	11	0.003	0.28
=> passed	12	0.005	0.25
=> passed	13	0.007	0.19
=> passed	14	0.015	0.31
=> passed	15	0.030	0.29
=> passed	16	0.046	0.19
=> passed	17	0.124	0.43
=> passed	18	0.250	0.31

Test 16-30: time isRamanujan(), for random Ramanujan numbers between 1 and 10^k,
for k = 4 to 18

The approximate order-of-growth is $n^{(\log \text{ratio})}$

	k	seconds	log ratio

=> passed	4	0.000	
=> passed	5	0.000	
=> passed	6	0.000	
=> passed	7	0.000	
=> passed	8	0.000	
=> passed	9	0.000	
=> passed	10	0.000	
=> passed	11	0.000	
=> passed	12	0.001	
=> passed	13	0.002	
=> passed	14	0.005	0.42
=> passed	15	0.009	0.24
=> passed	16	0.019	0.33
=> passed	17	0.030	0.20
=> passed	18	0.067	0.36

Test 31-37: time isRamanujan() method with random Ramanujan numbers
between 1 and 1000000

- reference solution calls per second: 637605.00
- student solution calls per second: 349534.00
- reference / student ratio: 1.82

=> passed student <= 10000x reference
=> passed student <= 5000x reference
=> passed student <= 1000x reference
=> passed student <= 500x reference
=> passed student <= 100x reference
=> passed student <= 50x reference
=> passed student <= 10x reference
=> BONUS student <= 2x reference

Test 38-44: time isRamanujan() method with random non-Ramanujan numbers
between 1 and 1000000

- reference solution calls per second: 150355.00
- student solution calls per second: 142237.50
- reference / student ratio: 1.06

=> passed student <= 10000x reference
=> passed student <= 5000x reference
=> passed student <= 1000x reference
=> passed student <= 500x reference
=> passed student <= 100x reference
=> passed student <= 50x reference
=> passed student <= 10x reference
=> BONUS student <= 2x reference

Ramanujan Total: 46/44 tests passed!

=====

* TIMING

Timing Inversions
*-----

Running 57 total tests.

Tests 1-10: calling generate(n, k) with k = 0 and n ranging
from 8192 to 4194304, going up by a factor of 2

The approximate order-of-growth is $n^{(\log \text{ratio})}$

	n	seconds	log ratio

	8192	0.000	
	16384	0.000	
	32768	0.000	
	65536	0.001	
	131072	0.001	
	262144	0.003	1.0
	524288	0.006	1.0
	1048576	0.012	1.0
	2097152	0.025	1.1
	4194304	0.041	0.7

==> passed

Tests 11-20: calling generate(n, k) with k = n(n-1)/2 and n ranging
from 8192 to 4194304, going up by a factor of 2

The approximate order-of-growth is $n^{(\log \text{ratio})}$

	n	seconds	log ratio

	8192	0.884	Infinity
	16384	3.485	2.0
	32768	13.974	2.0

Aborting test: exceeded time limit of 5.0 seconds

==> **FAILED**

Tests 21-30: calling generate(n, k) with k = random and n ranging from 8192 to 4194304, going up by a factor of 2

The approximate order-of-growth is $n^{(\log \text{ ratio})}$

n	seconds	log ratio
8192	0.158	Infinity
16384	1.607	3.3

WARNING: the time limit of 30 seconds was exceeded, so not all tests could be completed.

Total: 0/57 tests passed: Could not complete tests, which results in a reported score of 0.
=====

Timing MaximumSquareSubmatrix
*-----
Running 15 total tests.

Tests 1-15: Calling size() with random n-by-n matrices, with n ranging from 16 to 2048, going up by a factor of sqrt(2).

The approximate order-of-growth is $n^{(\log \text{ ratio})}$

	n	seconds	log ratio
=> passed	16	0.000	
=> passed	23	0.000	
=> passed	32	0.000	
=> passed	45	0.000	
=> passed	64	0.000	
=> passed	91	0.001	
=> passed	128	0.002	2.8
=> passed	181	0.003	1.2
=> passed	256	0.009	3.2
=> passed	362	0.012	0.7
=> passed	512	0.024	2.0
=> passed	724	0.050	2.1
=> passed	1024	0.100	2.0
=> passed	1448	0.201	2.0
=> passed	2048	0.396	2.0

MaximumSquareSubmatrix Total: 15/15 tests passed!

=====

* TIMING

Timing Ramanujan
*-----
Running 44 total tests.

Tests 1-15: time isRamanujan(), for random non-Ramanujan numbers between 1 and 10^k , for $k = 4$ to 18

The approximate order-of-growth is $n^{(\log \text{ ratio})}$

	k	seconds	log ratio
=> passed	4	0.000	
=> passed	5	0.000	
=> passed	6	0.000	
=> passed	7	0.000	
=> passed	8	0.001	
=> passed	9	0.001	
=> passed	10	0.002	
=> passed	11	0.005	0.49
=> passed	12	0.004	-0.07
=> passed	13	0.008	0.26
=> passed	14	0.017	0.35
=> passed	15	0.029	0.22
=> passed	16	0.065	0.35
=> passed	17	0.127	0.29
=> passed	18	0.119	-0.03

Test 16-30: time isRamanujan(), for random Ramanujan numbers between 1 and 10^k , for $k = 4$ to 18

The approximate order-of-growth is $n^{(\log \text{ ratio})}$

	k	seconds	log ratio
=> passed	4	0.000	
=> passed	5	0.000	
=> passed	6	0.000	
=> passed	7	0.000	
=> passed	8	0.000	
=> passed	9	0.000	
=> passed	10	0.000	
=> passed	11	0.000	
=> passed	12	0.000	
=> passed	13	0.002	
=> passed	14	0.005	0.45
=> passed	15	0.011	0.36
=> passed	16	0.018	0.21
=> passed	17	0.026	0.16
=> passed	18	0.043	0.22

Test 31-37: time isRamanujan() method with random Ramanujan numbers between 1 and 1000000
- reference solution calls per second: 655753.50
- student solution calls per second: 354723.50
- reference / student ratio: 1.85

=> passed	student <= 10000x reference
=> passed	student <= 5000x reference
=> passed	student <= 1000x reference
=> passed	student <= 500x reference
=> passed	student <= 100x reference
=> passed	student <= 50x reference
=> passed	student <= 10x reference
=> BONUS	student <= 2x reference

Test 38-44: time isRamanujan() method with random non-Ramanujan numbers between 1 and 1000000
- reference solution calls per second: 152458.50
- student solution calls per second: 129635.50
- reference / student ratio: 1.18

=> passed	student <= 10000x reference
=> passed	student <= 5000x reference
=> passed	student <= 1000x reference
=> passed	student <= 500x reference
=> passed	student <= 100x reference
=> passed	student <= 50x reference
=> passed	student <= 10x reference
=> BONUS	student <= 2x reference

Ramanujan Total: 46/44 tests passed!

=====